## Test Credentials

| Role | Username | Password |
|---|---|---|
| Admin | `admin` | `Admin@12345` |
| Booth Staff (Dhaka) | `dhaka_staff_1` | `Dhaka@12345` |
| Booth Staff (Chittagong) | `ctg_staff_1` | `Ctg@123456` |

---

## Step-by-Step Verification

### Prerequisites
1. **PostgreSQL** running on `localhost:5432` with database `bus_ticketing_dev`
2. **Node.js** and **Angular CLI** installed
3. **.NET 10 SDK** installed

---

### Step 1: Start the Backend API

```bash
cd /Users/prince/Downloads/BusTicketingSystem/src/Presentation/BusTicketing.Api
dotnet run
```

Expected: API starts on `http://localhost:5000` (or `https://localhost:5001`)

---

### Step 2: Verify Backend Health

Open in browser:
- `http://localhost:5000/health/live` → should return `Healthy`
- `http://localhost:5000/health/ready` → should return `Healthy` (once DB is up)
- `http://localhost:5000/scalar/v1` → API reference UI should load

---

### Step 3: Run Admin Frontend

```bash
cd /Users/prince/Downloads/BusTicketingSystem/frontend/bus-ticketing-admin
ng serve
```

```

Open `http://localhost:4200`

**Test as Admin (`admin` / `Admin@12345`):**
1. **Dashboard** — verify summary tiles show real aggregated data (Total Seats, Sold Seats, Available Seats, Today's Sales)
2. **Ticketing → Sell Ticket** — select a route/date, pick a trip, select a seat, fill passenger details, confirm sale → ticket stub appears with ticket number
3. **Ticketing → Search & Cancel** — search by ticket number, verify results, cancel a ticket → toast confirms cancellation, seat becomes available
4. **Ticketing → Payments** — verify payment list loads with Pending/Captured/Refunded statuses, test Capture/Refund/Fail actions
5. **Buses / Routes / Stations / Schedules / Users / Roles** — verify CRUD works as before

**Test as Booth Staff (`dhaka_staff_1` / `Dhaka@12345`):**
- Verify Users and Roles tabs are hidden (admin-only)
- Verify Sell, Search, Cancel, Dashboard still work

---

### Step 4: Run Client Frontend

```bash
cd /Users/prince/Downloads/BusTicketingSystem/frontend/bus-ticketing-client
ng serve
```

Open `http://localhost:4201`

**Test as Customer (register new account or use booth staff credentials):**
1. **Register** — create a new customer account → auto-login
2. **Search Trips** — enter a travel date, optionally enter origin/destination → results show available trips
3. **Select Seats** — click "Select Seats" on a trip → seat grid loads, click multiple seats (up to 10), fill passenger details, confirm booking → success toast with ticket numbers, redirect to My Tickets
4. **My Tickets** — verify booked tickets appear with correct status, verify Cancel button works for non-departed tickets, verify departed tickets show cancellation disabled
5. **Login/Logout** — verify auth flow, token refresh, redirect to login when accessing `/my-tickets` or `/booking` while logged out

---
```

### Step 5: Verify Security Hardening

**Rate Limiting:**
- Send 6+ failed POST requests to `/api/v1/auth/login` within 1 minute from the same IP
- 6th request should return HTTP 429 with `{"title":"Too many requests."}`

**Security Headers:**
```bash
curl -sI http://localhost:5000/health/live | grep -i
"X-Content-Type-Options\|X-Frame-Options\|Strict-Transport-Security"
```
Expected headers present (in non-Development mode, or when request is not HTTPS).

**Permission Policies:**
- Login as `dhaka_staff_1` (Booth Staff)
- Try `POST /api/v1/users` → should return 403 Forbidden
- Try `GET /api/v1/dashboard/summary` → should return 200 (DashboardView permission)
- Try `POST /api/v1/booking/tickets` → should return 201 (BookingSell permission)

**Double-Booking Prevention:**
- Open two browser windows, both as admin
- Both select the same trip and same seat
- First clicks Confirm → succeeds
- Second clicks Confirm → gets 409 Conflict toast, seat map refreshes

---

### Step 6: Verify Ticket Number Uniqueness

- Sell 3 tickets for the same date
- Verify ticket numbers are sequential: `TKT-YYYYMMDD-0001`, `TKT-YYYYMMDD-0002`, `TKT-YYYYMMDD-0003`

---

### Step 7: Run Unit Tests

```bash
cd /Users/prince/Downloads/BusTicketingSystem
dotnet test tests/BusTicketing.UnitTests/BusTicketing.UnitTests.csproj
```

Expected: `Passed! - Failed: 0, Passed: 43, Skipped: 0`

---

### Step 8: Verify Frontend Builds

```bash
cd /Users/prince/Downloads/BusTicketingSystem/frontend/bus-ticketing-admin && ng build
cd /Users/prince/Downloads/BusTicketingSystem/frontend/bus-ticketing-client && ng build
```

Both should complete with `Application bundle generation complete.` and no errors.

---

## Known Limitations

- **EF Core migrations**: Not yet generated. Run `dotnet ef migrations add InitialCreate` from the `BusTicketing.Infrastructure` project once `dotnet ef` is available, then `dotnet ef database update`.
- **Integration tests**: 4 tests fail because they require a running PostgreSQL instance. The 2 health-check tests pass.
- **CORS**: Currently allows all origins. Tighten in `Program.cs` for production.
- **localStorage tokens**: Documented in `SECURITY.md` as needing httpOnly cookie migration before public exposure.

—----------------------------------------------

```
# Security

## Authentication & session model

- **Access tokens**: JWT, HMAC-SHA256 signed, 15-minute default expiry
 (`Jwt:AccessTokenExpiryMinutes`), carrying `sub`, `unique_name`, `email`,
 `role`, `fullName`, and optionally `booth` claims. Validated on every request via
 `TokenValidationParameters` (issuer, audience, signing key, lifetime, 30s clock skew).
- **Refresh tokens**: opaque 64-byte random values (`RandomNumberGenerator`), 7-day
```

default expiry, stored server-side per user, **single-use with rotation**: every
successful refresh revokes the token used and issues a new pair (`RefreshTokenCommandHandler`). Replaying an already-revoked token is treated as a
theft signal — the entire active token chain for that user is revoked immediately.
- **Logout** revokes the specific refresh token supplied; it does not (and cannot,
statelessly) invalidate an already-issued access token before its natural
15-minute expiry — a standard, documented tradeoff of JWT bearer auth. Keeping
the access token lifetime short is the mitigation.

## Password storage

PBKDF2-HMACSHA256, 210,000 iterations (OWASP's 2023+ minimum recommendation for this
algorithm), 16-byte random salt per password, 32-byte derived key, constant-time
comparison (`CryptographicOperations.FixedTimeEquals`) on verification. Iteration
count travels with the hash (`{iterations}.{salt}.{hash}`) so it can be raised in
future without invalidating existing hashes — see `Pbkdf2PasswordHasher`.

**Candidate upgrade**: Argon2id is the stronger modern default for greenfield systems
with no legacy-hash constraint. PBKDF2 was chosen here for zero additional native
dependency; noted in ARCHITECTURE.md as a deliberate, revisitable tradeoff.

```
## Authorization

Role-based via ASP.NET Core's `[Authorize(Roles = "Admin")]` on
a per-endpoint basis
(see API.md for the full matrix). Two system roles are seeded
and protected from
modification/deletion at the domain layer (`Role.IsSystemRole`):
`Admin`, `BoothStaff`.
Custom roles can be created by an Admin but carry no additional
endpoint-level
permissions in this phase (`Role.Description` is descriptive
only) — see ROADMAP.md
for planned fine-grained permission claims.

## Token storage on the frontend

The Angular app stores both tokens in `localStorage`
(`AuthService`). This is a
**known, documented tradeoff** for an MVP SPA:

| Storage | XSS exposure | CSRF exposure | Survives reload |
|---|---|---|---|
| `localStorage` (current) | Yes — any injected script can read
it | No | Yes |
| httpOnly cookie (harder alternative) | No | Yes (needs
anti-CSRF token) | Yes |

Given this is an internal booth-staff/admin tool (not a
public-facing app taking
arbitrary user content), the primary mitigation is disciplined
output encoding
(Angular's default template binding already HTML-escapes
interpolated values, so
```

```
reflected-XSS surface is low) rather than eliminating the risk
architecturally.
**Before this becomes public-facing or handles payment data
(Booking/Payment phase),
migrate refresh-token storage to an httpOnly, `SameSite=Strict`
cookie** issued by the
API, with the access token kept in memory only (not persisted) —
this is called out
explicitly in ROADMAP.md as a prerequisite for the next phase,
not an afterthought.

## Transport & headers

- `UseHttpsRedirection()` active outside Development.
- CORS restricted to explicitly configured origins
(`Cors:AllowedOrigins`), not
 wildcarded — see DEPLOYMENT.md; the recommended production
topology (nginx
 reverse-proxying the SPA and API same-origin) avoids needing
CORS at all.
- Standard security headers (HSTS, X-Content-Type-Options, etc.)
are not yet
 explicitly configured beyond ASP.NET Core defaults — flagged in
ROADMAP.md as a
 pre-production hardening item (e.g. via `NWebsec` or manual
middleware).

## Input validation

Every command has a FluentValidation validator executed by
`ValidationBehavior`
before the handler runs — no handler trusts unvalidated input.
Validation failures
```

never reach the database layer or throw unhandled exceptions; they return a
structured `Result.Failure(Error.Validation(...))` mapped to HTTP 400 with a
field-level error map.

## Audit trail

Every create/update/status-change/login writes an `AuditLog` row (`IAuditLogService.LogAsync`) capturing the acting user, action, entity, and
timestamp, in the same database transaction as the business change it records —
see DATABASE.md and ARCHITECTURE.md for why this can never diverge from the actual
change.

## Known gaps for a public-facing / production deployment

Documented explicitly rather than silently absent:

1. No rate limiting on `/auth/login` (brute-force mitigation) — planned via
   `Microsoft.AspNetCore.RateLimiting` in ROADMAP.md.
2. No account lockout after repeated failed logins.
3. No MFA.
4. Refresh token storage should move to httpOnly cookies before public exposure (above).
5. Secrets in `appsettings.Development.json` are placeholder dev-only values —
   verified they are not production-suitable; see DEPLOYMENT.md's checklist.

—-- Add Migration to BusTicketingSystem Command
cd /Users/prince/Downloads/BusTicketingSystem/src/Infrastructure/BusTicketing.Infrastructure && dotnet ef migrations add InitialCreate --startup-project ../Presentation/BusTicketing.Api/BusTicketing.Api.csproj
—-- Update database to BusTicketingSystem Command